

Lab 09 - HR and multi-agent handoffs

AI Agents Fundamentals for Beginners | C690 | v1.0

Objective

Coordinate two bounded specialist roles and reconcile their outputs.

Alignment: LO4. Scheduled hands-on time: 35 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- `staff-roles.csv`
- `hr-policies.csv`
- `handoff-template.json`

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Open the synthetic staff role and policy files. Note which policy applies to operations and which applies only to managers.
2. Run `python3 run.py` and inspect the role-scoped checklist baseline. Confirm unrelated restricted fields are absent.
3. Create a fresh LLM conversation for the policy-research role using prompt 1. Ask for a source-cited onboarding checklist for the operations assistant.
4. Create a second conversation for the access-review role using prompt 2. Pass only the handoff envelope and relevant outputs, not unrelated staff records.
5. Use prompt 3 as coordinator to compare the two outputs. Resolve disagreement using policy version and role scope rather than majority vote.
6. Save both specialist outputs, the handoff envelope and final checklist. Identify a single accountable HR reviewer and one escalation condition.

Acceptance checks

- Operations staff do not receive manager-only approvals.
- The handoff preserves `task_id` and source references.
- The final checklist resolves or escalates conflicting evidence.

Run `python verify.py` after `run.py` (use `python3` if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's `requirements.txt`.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.

- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after the lab; keep source data and prompts unchanged.